ALGORITHMS II | GROUP PROJECT

Smart Delivery & Logistics Network

Computational geometry and approximation algorithms for delivery analysis

Group members

Akash Chakraborty

Registration no. 1059 Roll no. CSE/23011

Anipriyo Mandal

Registration no. 1065 Roll no. CSE/23017

Arpan Mandal

Registration no. 1075 Roll no. CSE/23027

Smart Delivery & Logistics Network

Problem statement and algorithmic solution | Algorithms II project

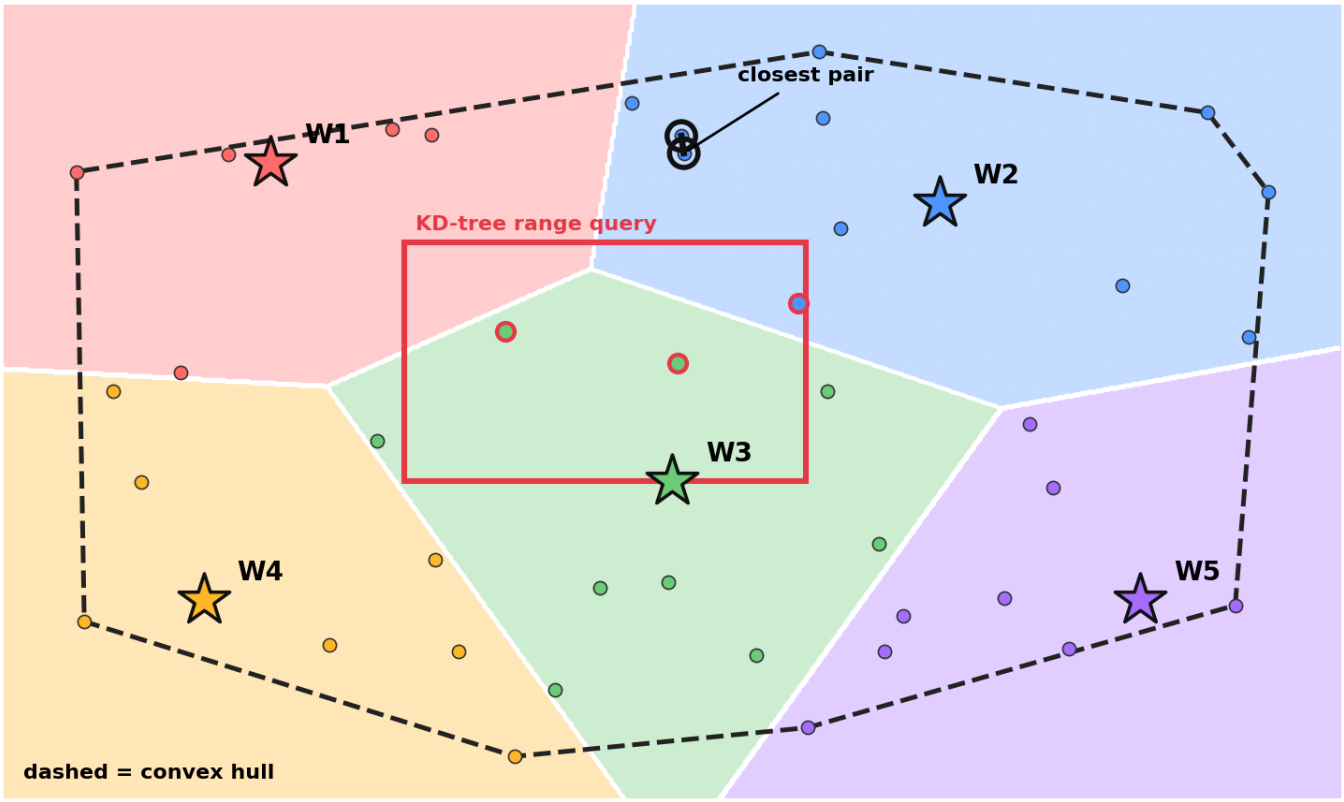
Voronoi diagram - KD-tree range search - Closest pair - Convex hull - Greedy set cover

The problem

A delivery company runs several warehouses and serves hundreds of customer stops. Once the list grows past what anyone can eyeball, four questions get hard: **which warehouse should serve which customers**, **which stops sit close together**, **what lies inside a given area**, and **how far the network actually reaches**. Answering them by comparing every point with every other point takes quadratic time and stops being practical quickly.

Our approach

We treat every warehouse and customer as a point on a 2D plane (ID, x, y, type) and give each question to the Algorithms II technique built for it. The results feed one dashboard. The project is a **geometric decision-support tool**, not a full vehicle-routing system: no traffic, fuel or driver scheduling.



Demo run: 5 warehouses (stars), 40 customers. Colours are Voronoi service areas, the dashed line is the convex hull, the red box is a range query (3 customers found), the circled pair is the closest pair (2.18 units apart).

Pipeline

Load data	Convex hull	KD-tree	Closest pair	Voronoi	Dashboard
warehouses + customers	network boundary	area queries	nearby stops	service areas	summary + maps

Which algorithm answers which question

Algorithm	Question it answers	How we use it	Cost
Voronoi diagram	Which warehouse is nearest to each part of the delivery area?	Build the diagram of the warehouse sites (Fortune's sweep). Each cell is one warehouse's service region, and every customer is assigned to the cell it falls in.	O(m log m) for m warehouses
KD-tree and range search	Which customers are inside this rectangle?	Build a KD-tree over customers, alternating x and y splits. A rectangle query skips whole subtrees that miss the box instead of testing every point.	build O(n log n), query O(√n + k)
Closest pair	Which two stops are nearest each other?	Divide and conquer: split by x, solve both halves, then check only the thin strip around the dividing line. Run it per warehouse region to spot stops worth grouping.	O(n log n) vs $O(n^2)$ brute force
Convex hull (Graham scan)	How far does the network reach?	Sort by angle around the lowest point and pop any point that makes a right turn. What is left is the smallest convex polygon holding all customers.	O(n log n)
Greedy set cover (optional)	Which few warehouses cover the most demand?	Each candidate site covers the customers within a radius. Repeatedly pick the site covering the most still-uncovered customers.	ln n approximation ratio

How the pieces work together

Voronoi + KD-tree. Voronoi tells us which warehouse owns each customer. The KD-tree then pulls out the customers in any rectangle the user draws, and their warehouse labels come along, so one selection shows both who is there and who serves them.

Voronoi + closest pair. Inside one warehouse region we run closest pair on that region's customers. In the demo run the nearest two stops are 2.18 units apart, which flags them as candidates for a shared delivery. We describe this as proximity analysis only: closest pair does not solve routing by itself.

Convex hull + everything. The hull gives the outer edge of the customer set (8 of the 40 demo points sit on it), which shows the reach of the network and any warehouse that sits far from its own boundary.

Worked example from the demo run

Step	Input	Output
1. Convex hull	40 customer points	Boundary of 8 points, drawn as the dashed outline
2. Voronoi	5 warehouses W1 to W5	5 service regions; each customer labelled with its nearest warehouse
3. KD-tree range query	Box x 30 to 60, y 40 to 70	3 customers returned without scanning all 40
4. Closest pair	All 40 customers	One pair, 2.18 units apart

Optional extension: choosing which warehouses to keep

Suppose the company has four candidate sites but wants to run as few as possible. Each site covers a set of customers within its service radius, and we want every customer covered. Finding the true minimum is NP-hard, so we use greedy set cover, which is guaranteed to be within a factor of about ln n of the best answer.

Candidate	Covers customers
W1	1, 2, 3, 4
W2	3, 4, 5, 6, 7
W3	6, 7, 8
W4	1, 2, 8, 9

Round 1: W2 covers 5 new customers, the most, so pick it. Customers 3 to 7 are now covered.

Round 2: W4 covers 4 of the 4 that remain (1, 2, 8, 9), so pick it.

Result: W2 and W4 cover all nine customers. Two sites instead of four.

The dashboard

Load data CSV of warehouses and customers	**Show network** plot all points	**Convex hull** outer boundary	**Voronoi** warehouse service regions
Range search draw a box, list deliveries	**Closest pair** highlight the nearest two	**Coverage** greedy set cover (optional)	**Summary** counts per region, hull size, distances

Scope, build order and limits

Build order

1. Point input and plotting.
2. Convex hull (Graham scan).
3. Closest pair.
4. KD-tree and range query.
5. Voronoi.
6. Dashboard wiring.
7. Greedy set cover, only if time allows.

Not in scope

Vehicle routing, traffic prediction, GPS navigation, fuel optimisation, driver scheduling. They would pull the project away from the Algorithms II syllabus.

Known limits

Straight-line (Euclidean) distance stands in for road distance, and coordinates are a flat 2D plane. Real GPS data could be mapped onto the same code later.

In one line: Voronoi decides who serves whom, the KD-tree finds what is where, closest pair finds what is near, the hull shows how far it all reaches, and set cover picks the sites worth keeping.